



LIONS LIGHT ACADEMY

Technical Operations Handbook

How the website and the Google backend actually work

*"It is the glory of God to conceal a matter, but the glory of kings is to
search it out."*

PROVERBS 25:2

EFFECTIVE

August 29, 2026

DOCUMENT

Version 1.0

Who this is for. Whoever maintains Lions Light Academy's website and Google account — today that is Damian, tomorrow it may not be. Everything here was learned by building the thing or by breaking it. Read Part Six before you change anything.

What is deliberately not here. Passwords, class join codes, and account credentials. Those live in the maintainer's own records, never in a published document. This handbook tells you what exists and where; it does not hand anyone the keys.

PART ONE • The shape of the thing

Lions Light Academy runs on **one HTML file and a Google account**. That is the whole architecture, and it is worth understanding why before you are tempted to improve it.

LAYER	WHAT IT IS
Website	A single <code>index.html</code> — roughly 1,000 lines, with all CSS and JavaScript inline
Hosting	Cloudflare Pages, deployed automatically from GitHub
Build step	None. The HTML is served exactly as committed
Database	None. The only stored data is a Google Sheet of prayer requests
Backend	Google Workspace — Forms, Sheets, Apps Script, Calendar, Classroom, Chat

There is no framework, no bundler, no server, and no staging environment. **A push to `main` publishes to production.** That sounds reckless and mostly isn't: with no build step there is nothing to break between commit and deploy, and a bad commit is reverted in seconds.

The cost of this design is that **every configuration value is hard-coded in `index.html`**. There is no `.env` file and no Cloudflare environment variable. Class links, form URLs, the calendar ID, the Chat space, the contact email — all of it is literal text in one file. That is the single most important thing to know before editing.

PART TWO • Google Classroom

The ownership model, and why it matters

All ten classes are owned by `lionslightacademy2025@gmail.com` . Every teacher, including Damian, is a co-teacher.

That is deliberate. In Google Classroom, whoever creates a class owns it permanently — ownership cannot be transferred. If a teacher creates their own class and then steps away mid-year, LLA cannot get into it, cannot recover the students' work, and the link on the website goes dead. Co-teachers can do everything that matters: post, assign, grade, add students, change the theme. The only things they cannot do are delete the class or remove the owner.

Never create a class from a personal account. The three minutes saved will cost a term's work eventually.

The ten classes

CLASS	SECTION	MEETS
E Science	K-5	Tuesday 9:00 – 9:45
E Art	K-5	Tuesday 10:15 – 11:00
E History	K-5	Tuesday 11:05 – 11:50
MS Art	5-8	Tuesday 9:00 – 10:00
MS History	5-8	Tuesday 10:15 – 11:00
MS Science	5-8	Tuesday 11:05 – 11:50
HS Chemistry	9-12	Tuesday 9:00 – 10:20
HS Computer Science	9-12	Tuesday 10:30 – 11:50
HS Real World Ready	9-12	Tuesday 12:30 – 1:30
Drama and Public Speaking	K-8	Tuesday 12:30 – 1:30

`HS Biology` and `HS World History` were archived at the end of 2025–2026. Archived is not deleted — both are recoverable from **Archived classes** if a subject returns.

Class links are class IDs, not join codes

This is the trap that cost the most time, so it gets its own heading.

A Google Classroom class has **two** identifiers, and they are not interchangeable:

- A **class ID** — a long string like `0DE5MzE4NTUyMzEw`. This is what belongs in a website link:
`classroom.google.com/c/<CLASS_ID>`
- A **join code** — eight lowercase characters, in the shape `a1b2c3d4`. Students type this into **Join class**. It is closer to a password than an address, which is why no real one appears in this document

For most of the site's life, all nine class links were built from **join codes pasted into** `/c/` URLs. Every one of them opened a blank page — no error message, no 404, just a spinner that never resolved. Nobody noticed because a broken Classroom link looks exactly like a slow one.

The rule: if a class link in `index.html` contains eight lowercase characters, it is wrong. Real class IDs are long and mixed-case.

Get the real link by opening the class and copying it out of the browser address bar, or from **Settings → Invite link**.

Two Google behaviours that will waste an hour if you don't know them

Creating a class re-prompts an attestation every single time. Google shows *"Using Classroom at a school with students?"* with a checkbox asserting you are **not** using Classroom at a school with students. It is not once per account or once per session — it fires on every class creation. It is also a legal assertion about what LLA is, so the person who owns the organisation should be the one clicking it, not a helper.

Co-teacher invitations exist only in email. They do not appear inside Classroom — no card, no banner, no pending notice. Until a teacher accepts from the email, `classroom.google.com` shows them an empty page reading *"Add a class to get started,"* exactly as though nothing had been sent. Every teacher will assume they were skipped. **Tell them in advance**, and repeat it in the message that announces the invitations.

Getting students in

Do not collect email addresses. Hand out the class code on the first day and have students go to **classroom.google.com → + → Join class**. Five minutes of class time, versus twenty email addresses typed by hand.

A student without their own device: give the code to a parent, who joins on their own device and sees everything the student would. They appear on the roster under the parent's name rather than the student's, so note who is who before grading.

One limitation, permanently

LLA runs on **personal Google accounts**, not Google Workspace for Education. Two consequences:

- **Guardian summaries are unavailable.** Google's automatic parent digest emails only work on Workspace for Education. Do not promise families a digest that cannot be sent
- Personal accounts cap at 250 members per class and 100 invitations per day — nowhere near a constraint at this size, but it is the ceiling

If LLA ever incorporates and qualifies for Workspace for Education, guardian summaries become available and this is worth revisiting.

PART THREE · Google Chat

The space is **Lions Light Academy**, owned by the LLA account. Its link appears in **five places** in [index.html](#) : Quick Links, onboarding Step 2, Student Spotlight, Snow Day Connect, and the footer.

Before August 2026 those five links pointed at a room ID that was actually the MS History **class join code** — the same copy-paste error as the Classroom links. No such room ever existed.

There is no self-serve join

Opening the space link as a non-member shows an empty Chat home with no join prompt and no preview. **Members must be added by hand:** open the space → its name at the top → **Manage members** → **Add people**.

The link on the website is a shortcut for people already in the room, not an invitation. That is why the site's onboarding copy says families are *added when they register* rather than telling them to "join" — the earlier wording promised something the link could not deliver.

A consumer Chat space has no access settings

Space details offers exactly three fields: name, description, guidelines. "Who can join" and "discoverable" are Google Workspace features and **do not exist on a personal account**. Do not go looking for them.

Keep history on

With history off, messages self-delete after 24 hours. A snow-day notice posted Monday night would be gone by Wednesday.

PART FOUR · The rest of the Google layer

SERVICE	WHAT IT DOES	NOTES
Google Forms	Enrollment application and prayer requests	Two form IDs hard-coded in <code>index.html</code> . Generator scripts are in the build kit
Google Sheets	Prayer form responses	The only datastore in the entire project
Apps Script	A <code>doGet</code> web app returning prayer JSON	The site's only live data call. If it fails the page degrades silently — the board just stays empty
Google Calendar	Embedded by <code>iframe</code>	This is the authoritative schedule. The Parent Packet points families here rather than at a schedule PDF, because block times vary by class and term

⚠ The calendar is a **group calendar not owned by the personal Google account**, and Google blocks both its ICS feed and its embed page to automated fetches. It cannot be read programmatically. Changes must be made by hand in the Calendar interface.

⚠ The Slack link on the site points at `slack.com/signin`, a generic login page. **It is not a workspace invite and never was.** Anyone clicking it lands nowhere useful. Either create a real workspace and replace the link, or remove it.

PART FIVE · The website

Repository and deployment

- **Repo:** `github.com/damianbrockway-wq/lions-light-academy`, branch `main` only
- **Host:** Cloudflare Pages, publish directory is the repo root, no build command

- **Local:** `~/Documents/Claude/Projects/lla/site/`

Files live in two sibling folders and the distinction matters:

FOLDER	DEPLOYED?
<code>lla/site/</code>	Yes. Everything here is published
<code>lla/</code> (<i>the parent</i>)	No. Markdown sources, working files, build kit, retired documents

Deploy lag is real

GitHub can be current while the live page is stale for a minute or two. **To confirm content shipped, read the commit on GitHub.** Read the live page only to confirm deployment finished. Judging a push by refreshing the site produces false alarms.

Verify before every push

```
bash python3 - <<'EOF' import pathlib, re t = pathlib.Path("index.html").read_text() links =
sorted(set(re.findall(r'href="(docs/[^\"]+\.pdf)"', t))) for l in links: f = pathlib.Path(l)
print("OK " if f.exists() and f.open('rb').read(5)==b'%PDF-' else "BAD", l) print("orphans:",
{str(x) for x in pathlib.Path('docs').glob('*.pdf')}) - set(links)) EOF
```

Then `git add -A` — **not** `commit -am`, which silently skips new untracked files. A PDF built but never added is the classic way to ship a card pointing at nothing.

`_headers`

Cloudflare rules that force the **fillable** PDFs to download rather than open in the browser's inline viewer. This exists because Safari ignores the HTML `download` attribute for PDFs and traps users in a preview with no Download, no Print, and no Back.

Read-only PDFs are deliberately excluded so they preview naturally. Any new fillable form must be added to `_headers` or it will trap Safari users.

PART SIX • The PDF pipeline

Markdown in, branded PDF out, matching the site's visual identity. **This is a real asset. Do not rebuild it from scratch.** It ships in `build-pipeline.tar.gz` and inside the build kit.

How it works

1. Markdown → HTML via `python-markdown`
2. A navy cover page is generated from the document's metadata plus the gold logo
3. Rendered by headless Chromium via Playwright
4. **Cover and body are printed separately and merged with `pypdf`**, so the cover carries no page footer
5. Footers use Chromium's `footer_template` with page numbers

Fonts are Fraunces (display) and Inter (body), bundled with the pipeline. If you re-download them, note that the Google Fonts CDN is blocked in some sandboxes; `raw.githubusercontent.com/google/fonts` works.

Gotchas that have already shipped bugs to production

`strip_front()` must preserve leading blockquotes *and* the blank lines between them. Two separate bugs have come from this one function. First it ate the version-supersession callout on every document. Then it kept the callouts but dropped the blank lines separating them, fusing two blockquotes into one box with the sentences running together mid-paragraph.

Verify layout by rendering a page to an image, not by extracting text. This is the lesson worth carrying: fused paragraphs extract as perfectly valid prose and pass every string check you can write. A nine-point text-extraction test reported PASS on a page that was visibly broken.

`@page:first { margin:0 }` in `brand.css` exists for full-bleed covers. Any document built *without* a cover must override it, or page one loses all margins and content bleeds off the left edge.

pdfplumber drops characters in styled runs — colons, en dashes, parentheses. Never write case-sensitive or exact-substring PDF checks. Normalise whitespace first.

PART SEVEN • Security posture

Be honest about what is protected and what is not.

The parent portal is a client-side gate only. There is no server-side check. Treat everything behind it as public. **Never post student-identifiable information there** — not names attached to concerns, not

discipline notes, not anything a family would not want a stranger reading. Today the section holds placeholder text and the exposure is nil. It stops being nil the moment someone posts a real advisory about a real student, and the 2026 policy set generates exactly that kind of material.

If the portal ever needs to hold real content, move it behind Cloudflare Access or a Pages Function checking a server-side secret. Until then, treat it as public-but-quiet.

Class join codes are closer to passwords than addresses. Anyone holding one can join that class. Give them to students and parents directly. Do not publish them on the website, and do not put them in a document that goes on the website — including this one.

Everything in `index.html` is publicly readable, because it is a static file served as-is. Never put a secret in it and expect it to stay secret.

PART EIGHT · Known risks, carried forward

These are recorded so a future maintainer does not rediscover them the hard way.

ITEM	
	Insurance. LLA believes it is covered by the host church. It is an unaffiliated outside group, and that belief is very likely wrong. Verify in writing
	No legal entity, no EIN, no bank account. All money runs through an individual
	Field Trip Waiver names three consents that do not match the registration form's names
	HS Credit Summary and Maine Homeschool Checklist pre-fill different course plans
	Slack link is a dead end — see Part Four
	Google Chat has no self-serve join — every family must be added by hand
	The site never states where LLA meets , and the contact email appears only inside a floating button's code
	No custom domain, no 404 page, no <code>robots.txt</code> , no <code>sitemap.xml</code>

PART NINE · Common tasks

Change a published document. Edit its markdown source in `lla/`, bump the version in `build.py`, rebuild, copy the PDF into `site/docs/`, update the version label on its resource card in `index.html`, commit both together.

Add a class for a new year. Create it from the LLA account (expect the attestation prompt), invite the teacher as co-teacher, copy the **class link** from the address bar, paste it into the matching card in the Classes section.

Retire a class. Archive it — never delete. Then remove its link from `index.html`.

Add a family to Chat. Open the space → its name → Manage members → Add people. There is no other way.

Rebuild everything from nothing. Use `lla-build-kit.tar.gz`. It contains the site, the Apps Script sources, the PDF pipeline, every markdown source, and a `CONFIG.md` listing every hard-coded value with what to replace it with.

The one rule that prevents most problems. A fact lives in exactly one document, and everything else points at it. Every conflict ever found in this project came from one fact written in two places and updated in one.